

Practical 5

April 21, 2026

Zeal College of Engineering and Research
Subject: LP-VI

Name: Jenil Girish Rathod
Class: BE
Div: B
Batch: B1
Roll No: B22009

Group B: Practical No: 5

Program Statement: Perform the data classification using a classification algorithm or perform the data clustering using a clustering algorithm.

0.1 AIM: To perform data classification using a Decision Tree algorithm on a customer churn dataset.

0.1.1 STEP 1: Import libraries

In this step we import `pandas` for data handling and the necessary classes from `scikit-learn` for building and evaluating a Decision Tree classifier.

```
[1]: # Step 1: Import required libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

0.1.2 STEP 2: Load dataset

Read the Telco Customer Churn CSV into a pandas DataFrame and display the first five rows to inspect the data.

```
[2]: # Step 2: Load dataset
# The dataset is in the subfolder 'TelcoCustomerChurn' relative to this notebook
data_path = 'TelcoCustomerChurn/WA_Fn-UseC_-Telco-Customer-Churn.csv'
df = pd.read_csv(data_path)
print('Dataset loaded. Shape:', df.shape)
df.head()
```

Dataset loaded. Shape: (7043, 21)

```
[2]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No phone service	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No phone service	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	
4	No	No	No	Month-to-month	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]

0.1.3 STEP 3: Data preprocessing

We will: - Drop the customerID column (not a predictive feature). - Convert TotalCharges to numeric (it may be read as object). - Handle missing values. - Encode categorical columns using LabelEncoder.

```
[3]: # Step 3: Data preprocessing
# 1. Drop customerID if present
if 'customerID' in df.columns:
    df.drop('customerID', axis=1, inplace=True)

# 2. Convert TotalCharges to numeric (coerce errors to NaN)
if 'TotalCharges' in df.columns:
    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# 3. Inspect missing values
print('Missing values before handling:')
print(df.isnull().sum())
```

```

# 4. Drop rows with missing values (simple handling for this practical)
df.dropna(inplace=True)
print('\nShape after dropping missing rows:', df.shape)

# 5. Encode categorical/object columns using LabelEncoder
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le
# Show a sample after preprocessing
df.head()

```

Missing values before handling:

```

gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    11
Churn           0
dtype: int64

```

Shape after dropping missing rows: (7032, 20)

```

[3]:  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0      0              0        1            0        1             0
1      1              0        0            0       34             1
2      1              0        0            0        2             1
3      1              0        0            0       45             0
4      0              0        0            0        2             1

MultipleLines  InternetService  OnlineSecurity  OnlineBackup  \

```

0	1	0	0	2
1	0	0	2	0
2	0	0	2	2
3	1	0	2	0
4	0	1	0	0

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	0	0	0	0	0	
1	2	0	0	0	1	
2	0	0	0	0	0	
3	2	2	0	0	1	
4	0	0	0	0	0	

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	1	2	29.85	29.85	0
1	0	3	56.95	1889.50	0
2	1	3	53.85	108.15	1
3	0	0	42.30	1840.75	0
4	1	2	70.70	151.65	1

0.1.4 STEP 4: Split dataset

- Separate features (X) and target (y where target is Churn).
- Perform an 80/20 train-test split.

```
[4]: # Step 4: Split dataset into features and target
if 'Churn' not in df.columns:
    raise KeyError('Target column "Churn" not found in dataframe')

X = df.drop('Churn', axis=1)
y = df['Churn']

# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

print('X_train shape:', X_train.shape)
print('X_test shape:', X_test.shape)
print('y_train shape:', y_train.shape)
print('y_test shape:', y_test.shape)
```

```
X_train shape: (5625, 19)
X_test shape: (1407, 19)
y_train shape: (5625,)
y_test shape: (1407,)
```

0.1.5 STEP 5: Apply classification algorithm

Use a Decision Tree classifier and train it on the training data.

```
[5]: # Step 5: Train Decision Tree classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
print('Decision Tree model trained.')
```

Decision Tree model trained.

0.1.6 STEP 6: Prediction

Predict churn on the test set using the trained model.

```
[6]: # Step 6: Predict on test data
y_pred = model.predict(X_test)
print('Predictions completed. Example predictions:')
print(y_pred[:10])
```

Predictions completed. Example predictions:
[0 0 1 0 1 1 0 0 0 0]

0.1.7 STEP 7: Evaluation

Calculate and print the accuracy of the model on the test set.

```
[7]: # Step 7: Evaluation
acc = accuracy_score(y_test, y_pred)
print(f'Accuracy: {acc:.4f} ({acc * 100:.2f}%)')
```

Accuracy: 0.7249 (72.49%)

0.2 RESULT

- **Model outcome:** The Decision Tree model successfully classified customer churn on the Telco dataset.
- **Accuracy obtained:** Typically in the range of approximately 70%–80% for this baseline Decision Tree model (see printed accuracy above; exact value may vary by preprocessing and random seed).
- **Interpretation:** An accuracy around 70%–80% indicates the model captures useful patterns but is not perfect. Improvements can come from feature engineering, handling class imbalance, hyperparameter tuning (e.g., pruning the tree, setting `max_depth`), or using ensemble methods (Random Forest, Gradient Boosting). This notebook is intentionally simple and beginner-friendly for academic submission. You may expand it by adding cross-validation, a confusion matrix, or feature importance analysis.